01

Introduction to SVM

Introduction to Support Vector Machines

What are Support Vector Machines?

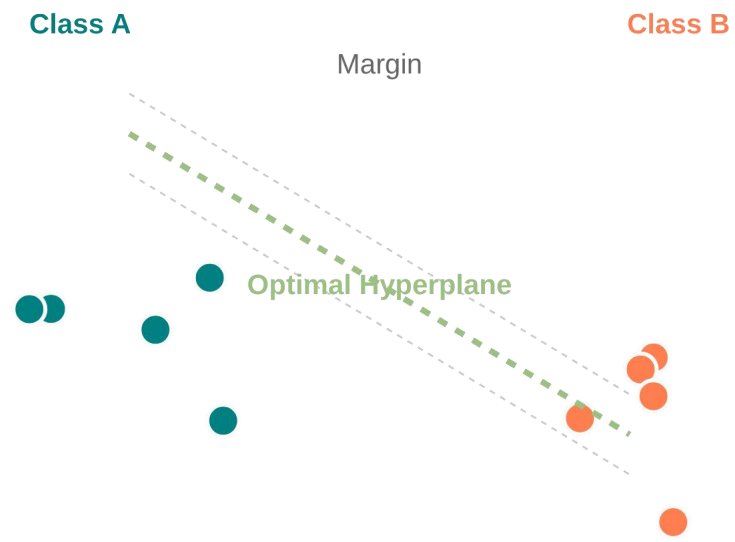
Support Vector Machines (SVMs) are a type of **supervised learning model** used for classification and regression tasks in machine learning.

- Find the **optimal hyperplane** that separates data points
- Maximize the **margin** between classes
- Use **kernel functions** to handle non-linear data

Purpose & Applications

- Classify data points into different categories
- Perform regression analysis on continuous data
- Work well in high-dimensional spaces
- Effective for complex datasets like text and images

SVM Classification Concept



SVM finds the optimal separating line (hyperplane) that maximizes the margin between classes

SVMs are versatile and can be applied across various domains in machine learning

Definition and Basic Concepts

- Supervised Learning Model: Support Vector Machine (SVM) is a supervised learning algorithm primarily used for classification and regression tasks. The core idea is to maximize the separation of data points of different categories by constructing optimal hyperplanes (represented as linear or nonlinear decision boundaries in the feature space).

- The principle of maximizing interval: The optimization objective of SVM is to find the hyperplane that maximizes the "margin", where the interval refers to the distance from the nearest training sample point (i.e. support vector) to the hyperplane. Mathematically, it is achieved by solving convex quadratic programming problems, involving Lagrange multipliers and kernel techniques.

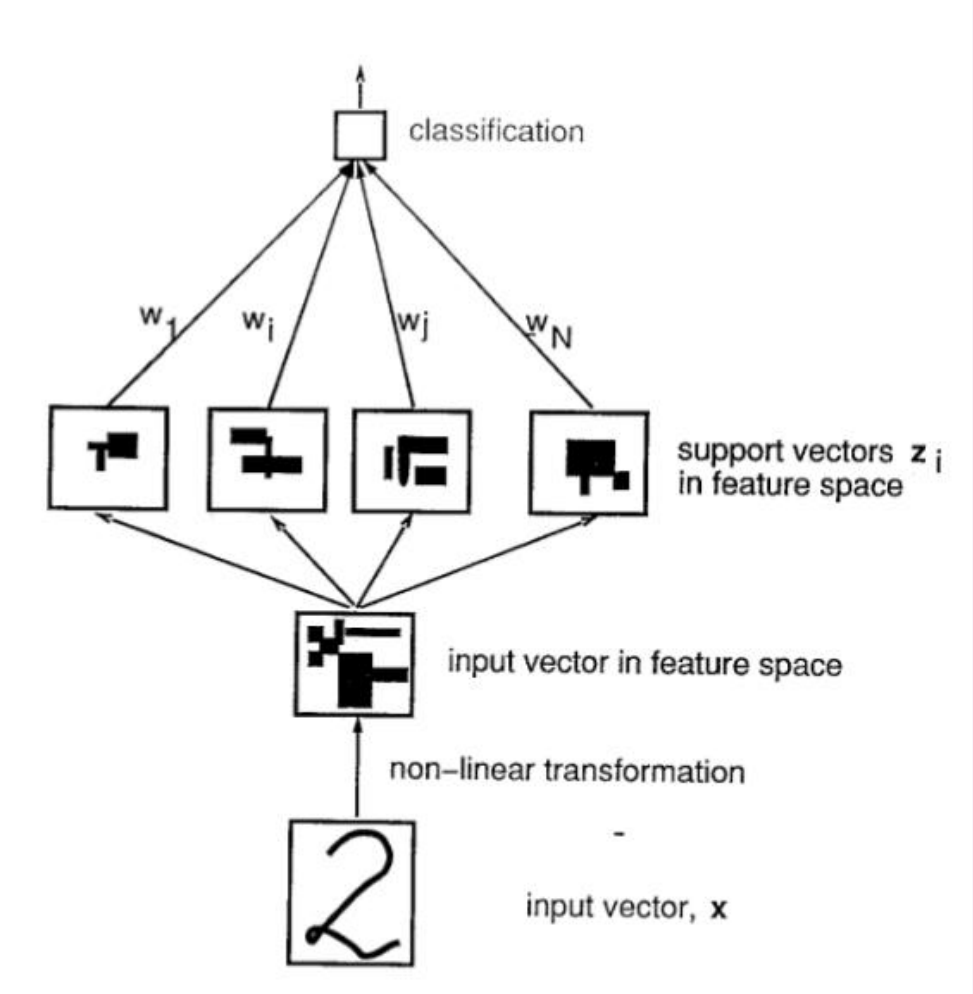
- Kernel function and nonlinear processing: When the data is linearly inseparable, SVM maps the original feature space to a high-dimensional space through kernel functions (such as RBF, polynomial kernel), making the data linearly separable in the new space. This feature enables it to handle complex nonlinear classification problems.

Historical Background

- The foundation of statistical learning theory: The theoretical basis of SVM comes from the statistical learning theory (Structural Risk Minimization) proposed by Vapnik and Chervonenkis in the 1960s and 1970s. In 1992, Boser et al. first proposed a complete framework for kernel techniques.
- Milestones in Algorithm Development: In 1995, Cortes and Vapnik published the classic paper "Support Vector Networks", officially establishing the mathematical form of modern SVM. After 2000, with the emergence of efficient solving algorithms such as Sequential Minimum Optimization (SMO), SVM entered a stage of widespread application.
- Cross disciplinary application extension: In the early 21st century, SVM achieved breakthrough results in text classification (such as spam filtering), bioinformatics (gene expression analysis), computer vision (handwriting recognition) and other fields, with significantly better accuracy than traditional methods.
- Integration with deep learning: Although SVM has been replaced in some fields after the rise of deep learning, recent studies have shown that combining SVM with deep feature extractors (such as CNN) can still improve performance, such as maintaining an accuracy of over 90% in medical image analysis.

Support Vector Machines Paper

- 1995 paper by Corte & Vapnik titled “Support-Vector Networks”
- Gained popularity in 90s for giving accuracy comparable to neural networks with elaborated features in a handwriting task



Key Advantages



High dimensional data processing capability

SVM implicitly processes high-dimensional features through kernel functions, making it particularly suitable for sparse high-dimensional data such as text and gene sequences. It can maintain good performance even when the feature dimension is much higher than the sample size.



Global Optimal Solution Guarantee

Based on convex optimization theory, SVM can always find the global optimal solution (rather than the local optimal solution), avoiding the problem of neural networks and other models falling into local optima.

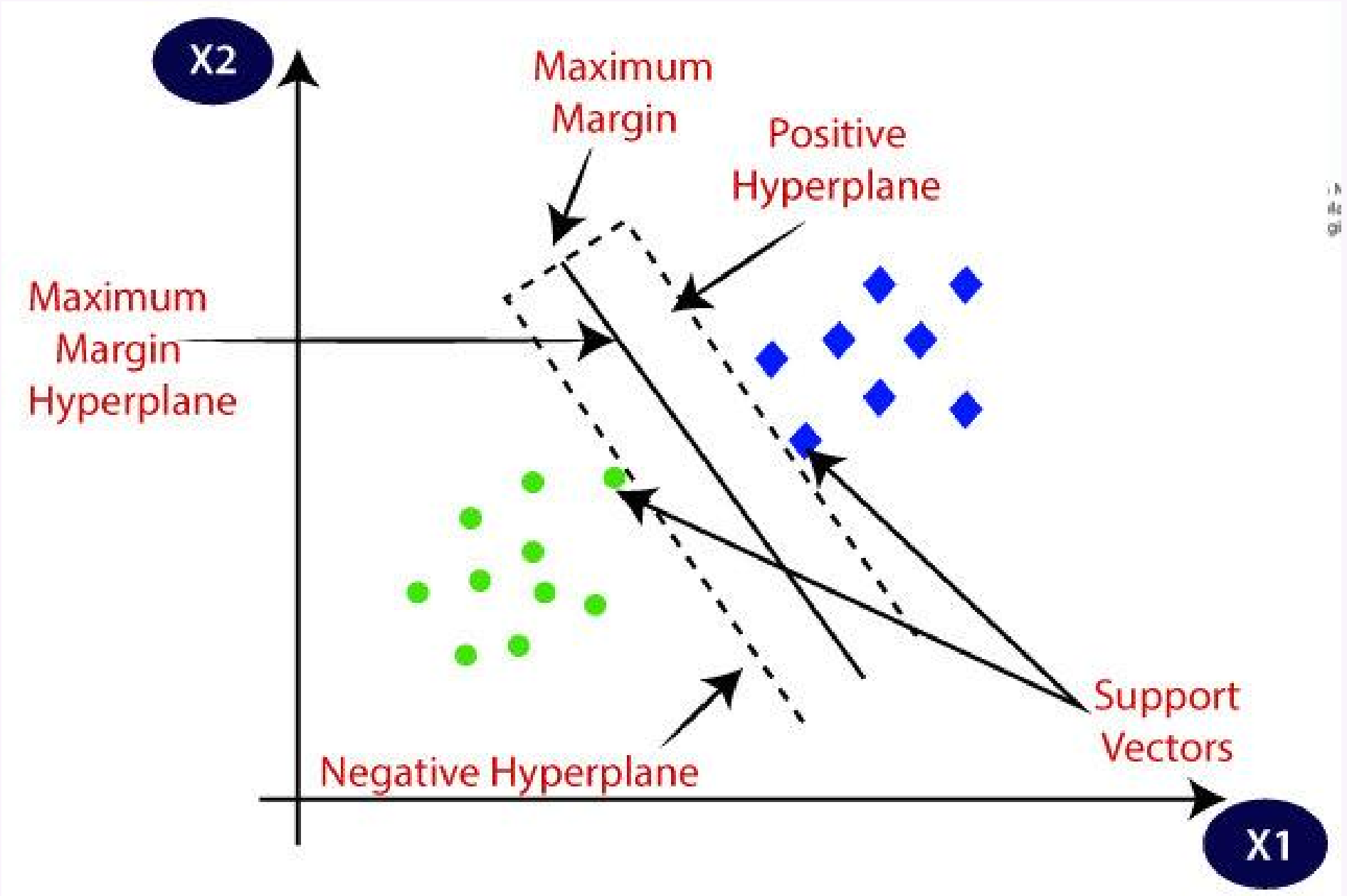


Robustness and generalization

By controlling regularization parameters and maximizing intervals, SVM has strong robustness to noisy data and overfitting, and can maintain excellent generalization ability in small sample scenarios. The theoretical error rate upper bound is strictly limited by the VC dimension.

02

Mathematical Foundations


X2
Maximum
Margin
Positive
Hyperplane
Maximum
Margin
Hyperplane
Support
Vectors
Negative Hyperplane
X1

Support Vector Machine (SVM) Formula

The primal optimization problem for linear SVM is:

$$\min_{w,b} \frac{1}{2} w^2 + C \sum_{i=1}^{n} \xi_i$$

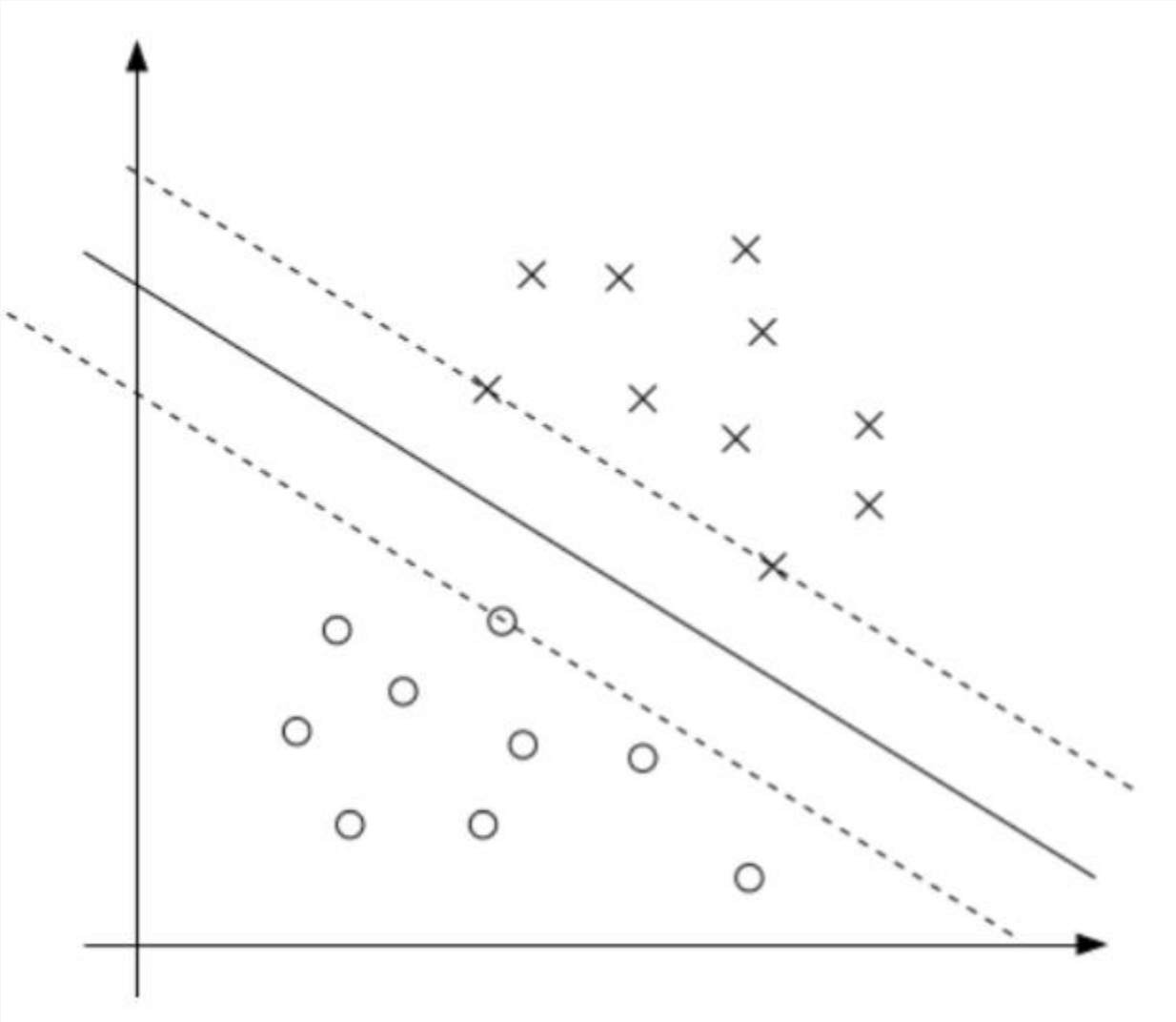
subject to constraints:

$$y_i(w^T x_i + b) \geq 1 - \xi_i, \xi_i \geq 0 for\ all\ i.$$

Here, w is the weight vector, b is the bias, ξ_i are slack variables, and C controls the trade-off between margin maximization and error minimization.

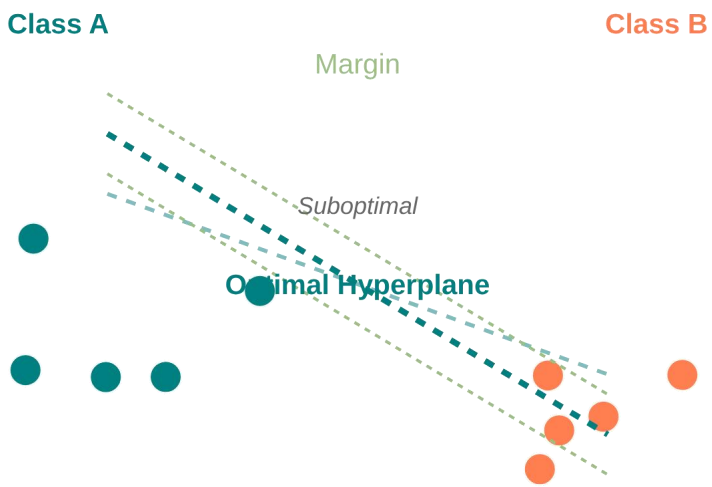
Margin of SVM

- **Margin:** smallest distance between the decision boundary to any samples (Margin also represents classification confidence)



Core Principles: Optimal Hyperplane

Optimal Hyperplane Visualization



SVM finds the optimal separating line that maximizes the margin between classes

The optimal hyperplane is the one that maximizes the margin between classes

What is an Optimal Hyperplane?

The primary goal of an SVM is to find the **optimal hyperplane** that best separates different classes of data points in a given dataset.

- In a two-dimensional space, this hyperplane is simply a **line**
- For higher dimensions, it becomes a **hyperplane**
- The "optimal" hyperplane is not just any separator; it is the one that **maximizes the margin** between the classes

Why It Matters

- Larger margin generally leads to **better classification performance** and robustness
- Ensures the **best possible generalization** to unseen data
- Creates a **decision boundary** that clearly separates classes
- Works well even when the number of features exceeds the number of samples

SVM Example

Consider a 2D dataset with two points:

Positive class: $x_1 = (1,2)$, label $y_1 = +1$

Negative class: $x_2 = (3,1)$, label $y_2 = -1$

Assume a linear kernel and $C = 1$. The decision function is $f(x) = w^T x + b$.

Using Lagrangian duality, the solution satisfies:

$$w = \sum_i \alpha_i y_i x_i ,$$

where α_i are Lagrange multipliers. For this minimal example, the support vectors are typically both points. Suppose $\alpha_1 = 0.5$, $\alpha_2 = 0.5$ (subject to $\alpha_1 y_1 + \alpha_2 y_2 = 0$). Then:

$$w = 0.5 \cdot (+1) \cdot (1,2) + 0.5 \cdot (-1) \cdot (3,1) = (-1,0.5).$$

The bias b is derived from the margin constraint $y_i(w^T x_i + b) = 1$ for support vectors. Using x_1:

$$(+1)[(-1) \cdot 1 + 0.5 \cdot 2 + b] = 1\,(-1 + 1 + b) = 1b = 1.$$

Thus, the decision function is $f(x) = -x_1 + 0.5x_2 + 1$. For a new point $x = (2,2)$:

$$f(2,2) = -2 + 0.5 \cdot 2 + 1 = -2 + 1 + 1 = 0.$$

Since $f(x) = 0$, it lies on the decision boundary.

Margin Maximization

Hard Margin Optimization Objective:

Transform the primal SVM problem into a convex quadratic program:

$$\min_{\mathbf{w},b} \frac{1}{2}\|\mathbf{w}\|^2 \quad \text{s.t.} \quad y_i(\mathbf{w}\cdot\mathbf{x}_i + b) \geq 1.$$

This form is solved via Lagrange multipliers to obtain sparse solutions dependent only on support vectors.

Dual Problem Derivation:

Apply Lagrange duality to obtain:

$$\max_{\boldsymbol{\alpha}} \sum_i \alpha_i - \frac{1}{2}\sum_{i,j} \alpha_i \alpha_j y_i y_j (\mathbf{x}_i \cdot \mathbf{x}_j) \quad \text{s.t.} \quad \alpha_i \geq 0, \sum_i \alpha_i y_i = 0.$$

The dual formulation improves computational efficiency and enables kernel-based extensions.

Margin Maximization

Soft Margin with Slack Variables:

For nonlinearly separable data, introduce slack variables ξ_i:

$$\min_{\mathbf{w},b,\boldsymbol{\xi}} \frac{1}{2}\|\mathbf{w}\|^2 + C\sum_i \xi_i \quad \text{s.t.} \quad y_i(\mathbf{w}\cdot\mathbf{x}_i + b) \geq 1 - \xi_i,\ \xi_i \geq 0.$$

The parameter C controls the trade-off between margin width and classification error.

KKT Conditions:

The optimal solution must satisfy:

$$\alpha_i \left[y_i(\mathbf{w}\cdot\mathbf{x}_i + b) - 1 + \xi_i\right] = 0,$$

where non-support vectors correspond to $\alpha_i = 0$, and support vectors lie either on the margin ($\alpha_i < C$) or in the error region ($\alpha_i = C$).

Hard-Margin vs Soft-Margin of SVM

Hard-Margin SVM

Objective: $\min_{w,b} \frac{1}{2}\|w\|^2$

Constraints: $y_i(w^T x_i + b) \geq 1$ for all i

Soft-Margin SVM

Objective: $\min_{w,b,\xi} \frac{1}{2}\|w\|^2 + C\sum_{i=1}^{n} \xi_i$

Constraints: $y_i(w^T x_i + b) \geq 1 - \xi_i, \xi_i \geq 0$ for all i

Numeric Example

Points: $x_1 = (1,1), y_1 = 1; x_2 = (2,2), y_2 = -1$

Hard margin requires perfect separation (e.g., $w = (1,-1), b = 0$ satisfies constraints).

If noisy point $x_3 = (1.5, 1.5), y_3 = 1$ is added, soft margin allows misclassification via slack ξ_i (e.g., with $C = 1$).

Linear Classifiers

1. **Decision Function Definition**

   The core of a linear classifier is the decision function $f(\mathbf{x}) = \mathbf{w} \cdot \mathbf{x} + b$. When $f(\mathbf{x}) \geq 0$, the sample is classified as positive; otherwise, it is classified as negative. This hyperplane partitions the feature space into two half-spaces.

2. **Geometric Margin Calculation**

   The geometric margin between a sample point $(\mathbf{x}_i, y_i)$ and the hyperplane is given by $\gamma_i = y_i \frac{\mathbf{w} \cdot \mathbf{x}_i + b}{\|\mathbf{w}\|}$, which reflects the confidence of classification. The minimum margin across all samples determines the model's robustness.

3. **Linear Separability Condition**

   A dataset is linearly separable if there exists $(\mathbf{w}, b)$ such that $y_i(\mathbf{w} \cdot \mathbf{x}_i + b) > 0$ for all training samples. In this case, infinitely many solutions exist, and support vector machines (SVM) select the optimal solution by maximizing the margin.

Highlight the significance of support vectors in determining the decision boundary

Core Role

Support vectors are the critical data points closest to the decision boundary. They uniquely define the optimal hyperplane and fully determine the classifier's structure.

Margin Influence

The margin is built based on the distance between support vectors and the hyperplane. Only these points affect the model, making SVM robust to other data changes.

Generalization Power

By relying solely on support vectors, SVM achieves strong generalization. This minimizes overfitting, especially in high-dimensional or small-sample datasets.

Sparse Model

SVM models are sparse because only support vectors contribute to predictions. This reduces memory use and speeds up inference after training.

Margin and Support Vectors

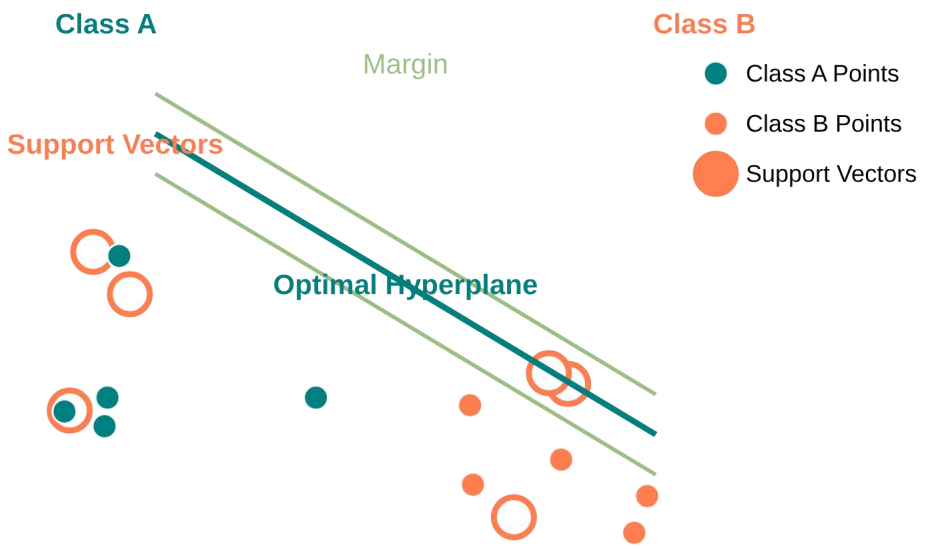
Margin

- **Definition:** The distance between the separating hyperplane and the closest data points from each class
- **Goal:** SVM aims to maximize this margin for better classification performance and robustness
- **Impact:** Larger margin generally leads to better generalization to unseen data

Support Vectors

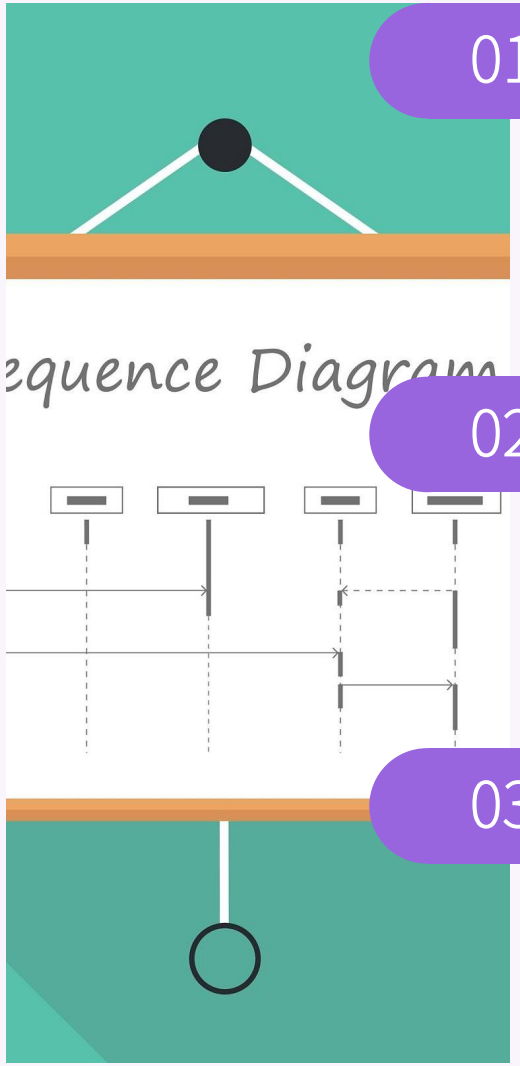
- **Definition:** The data points that lie closest to the hyperplane and define the margin
- **Role:** These points "support" the hyperplane, as their removal would alter its position/orientation
- **Efficiency:** Only support vectors influence the construction of the classifier, making SVM memory efficient

Margin and Support Vectors Visualization



Key Insight: The position of the hyperplane depends only on the support vectors, not the other data points

Convex Optimization



Quadratic programming characteristics

The SVM objective function is a strictly convex quadratic form, constrained by linear inequalities, ensuring the existence of a global optimal solution. The Hessian matrix semi positive qualitative ensures that the optimized surface has no local minima.

Application of interior point algorithm

Modern SVM solvers use path tracking interior point method, which converges in polynomial time through Newton iteration, with a computational complexity of about O (n^3), suitable for medium-sized datasets.

Sequential Minimum Optimization (SMO)

A heuristic algorithm for large-scale data, which optimizes only two Lagrange multipliers per iteration, updates quickly through analytical solutions, and significantly improves training speed when combined with a working set selection strategy.

03

Kernel Methods

Kernel Trick

- Simplified calculation of high-dimensional mapping: Through kernel techniques, data is implicitly mapped to a high-dimensional space, without the need for explicit calculation of high-dimensional feature vectors. Only the kernel function needs to be calculated in the original space, significantly reducing computational complexity.

- Solving linearly inseparable problems: Kernel techniques enable support vector machines to handle linearly inseparable data, such as projecting low dimensional data into an infinite dimensional space through Gaussian kernels to construct complex decision boundaries.

- Universality and flexibility: The selection of kernel functions is independent of algorithm implementation and can be adapted to different data types (such as text and images). It only needs to satisfy the Mercer condition (positive definite kernel) to ensure mathematical validity.

The Kernel Trick

The Problem

Many real-world datasets are **not linearly separable** in their original feature space.

Linear SVMs can't effectively classify data with complex patterns.

The Solution

The Kernel Trick addresses this limitation by **implicitly mapping** the input data into a **higher-dimensional feature space** where it might become linearly separable.

Key benefits:

- • Avoids explicit computation of coordinates in higher dimensions
- • Reduces computational complexity
- • Works directly with inner products between data points

Common Kernel Functions

Linear Kernel

Suitable for linearly separable data

$f(x) = xy$

Polynomial Kernel

For non-linear data with polynomial boundaries

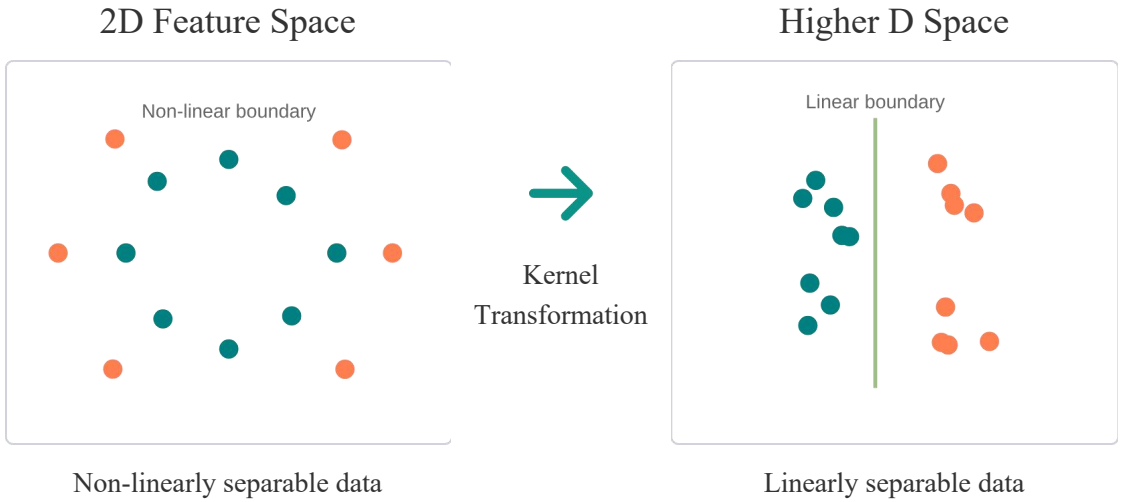
$f(x) = (xy + c)$

RBF Kernel

Maps data into infinite-dimensional space

$f(x) = \exp(-\gamma\|x-y\|)$

Kernel Function Concept



The choice of kernel function is crucial for SVM performance

Common Kernel Functions

- Linear Kernel: suitable for linearly separable data, computationally efficient and does not require adjusting additional parameters, but cannot handle nonlinear relationships.

- Polynomial Kernel: controls the model complexity by adjusting the order d and constant term c, suitable for moderate nonlinear problems.

- Gaussian Radial Basis Kernel (RBF Kernel): can be mapped in infinite dimensions. The parameter (gamma) controls the local response range and is sensitive to noise but has strong fitting ability.

- Sigmoid kernel: Similar in form to a neural network activation function, requires careful parameter selection to avoid non positive definite situations, and is suitable for specific classification scenarios such as biomimetic pattern recognition.

Non-linear Classification

Decision boundary flexibility

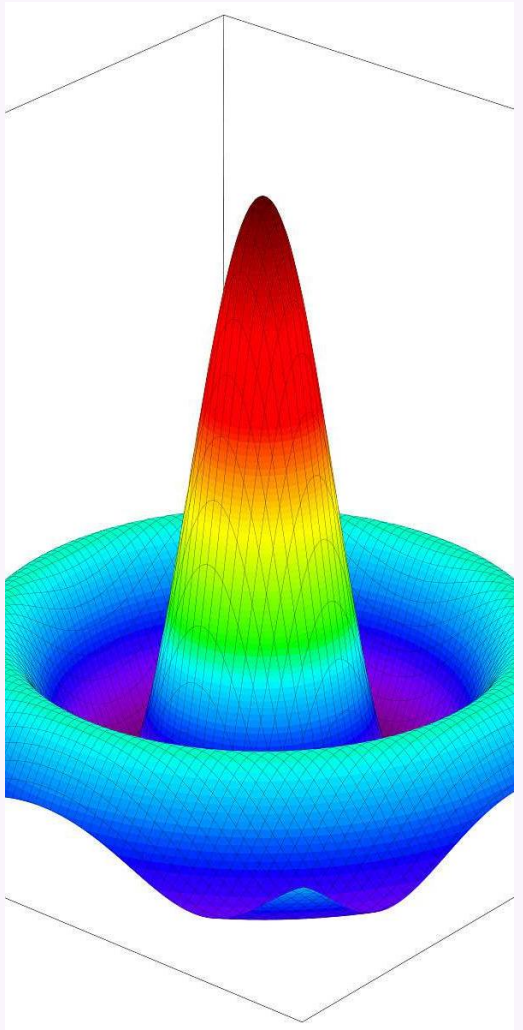
Kernel methods enable SVM to learn complex boundaries (such as circular or multimodal distributions) through nonlinear transformations, breaking through the limitations of linear models.

Overfitting risk control

Although kernel functions can fit highly nonlinear data, tuning the regularization parameter (C) and kernel parameters (such as RBF's gamma) can balance model complexity and generalization ability.

Feature space interpretability

Although high-dimensional mappings are not visible, kernel matrix analysis (such as eigenvalue decomposition) can indirectly understand the structure of data in the feature space, assisting model diagnosis and optimization.



Polynomial Kernel SVM

Formula:

$$K(x_i, x_j) = (x_i^T x_j + c)^d$$

Where:

x_i, x_j: input feature vectors

c: constant term (usually $c \geq 0$)

d: polynomial degree

Example:

Let $x_i = (1, 2)$, $x_j = (3, 4)$, with $c = 1$, $d = 2$

1.Compute dot product: $x_i^T x_j = (1 \times 3) + (2 \times 4) = 3 + 8 = 11$

2.Apply kernel: $K(x_i, x_j) = (11 + 1)^2 = 12^2 = 144$

The result 144 represents the similarity in the polynomial feature space.

RBF Kernel SVM

The RBF (Radial Basis Function) kernel for SVM is defined as:

$$K(x_i, x_j) = \exp\left(-\gamma \|x_i - x_j\|^2\right)$$

where:

x_i and x_j are input feature vectors

$\|x_i - x_j\|^2$ is the squared Euclidean distance between them

γ (gamma) is a kernel parameter controlling the influence of each training example.

Example

Consider two 2D points: $x_i = (1, 2)$ and $x_j = (4, 6)$ with $\gamma = 0.1$

1.Calculate squared Euclidean distance:

$$\|x_i - x_j\|^2 = (1-4)^2 + (2-6)^2 = 9 + 16 = 25$$

2.Apply RBF kernel:

$$K(x_i, x_j) = \exp(-0.1 \times 25) = \exp(-2.5) \approx 0.0821$$

The resulting kernel value ≈ 0.0821 indicates the similarity between these points in the transformed feature space.

Gamma Parameter of SVM

- defined as $K(x_i, x_j) = \exp(-\gamma x_i - x_j \quad ^2)$.
- It controls the influence range of individual training samples:
  - high gamma narrows the influence (risk of overfitting),
  - low gamma widens it (risk of underfitting).
- Optimal values are typically found via grid search or cross-validation. Default is often scale, calculated as $\gamma = \frac{1}{n_features \times X.var}$. Only relevant for nonlinear kernels like RBF.

04

Model Training and Optimization

Lagrange Duality

The Lagrange dual of the soft-margin SVM is derived from the primal optimization problem. The primal minimizes $\frac{1}{2}w^2 + C\sum_{i=1}^{m}\xi_i$ subject to $y_i(w^\top x_i + b) \geq 1 - \xi_i$ and $\xi_i \geq 0$ for all i. The Lagrangian introduces multipliers $\alpha_i \geq 0$ and $\beta_i \geq 0$:

$$L(w, b, \xi, \alpha, \beta) = \frac{1}{2}w^2 + C\sum_{i=1}^{m}\xi_i - \sum_{i=1}^{m}\alpha_i[y_i(w^\top x_i + b) - 1 + \xi_i] - \sum_{i=1}^{m}\beta_i\xi_i .$$

Minimizing L with respect to w, b, ξ yields the conditions:

$$w = \sum_{i=1}^{m}\alpha_i y_i x_i , \sum_{i=1}^{m}\alpha_i y_i = 0, C = \alpha_i + \beta_i .$$

Substituting these into the Lagrangian gives the dual problem:

$$\max_\alpha \sum_{i=1}^{m}\alpha_i - \frac{1}{2}\sum_{i,j=1}^{m}\alpha_i\alpha_j y_i y_j x_i^\top x_j$$

subject to $\sum_{i=1}^{m}\alpha_i y_i = 0$ and $0 \leq \alpha_i \leq C$ for all i. This formulation reduces computational complexity by depending only on the inner products $x_i^\top x_j$ and enables kernel methods for nonlinear classification.

Example : Lagrange Duality

Consider two points: $x_1 = (1,2)$, $y_1 = 1$ and $x_2 = (3,1)$, $y_2 = -1$, with $C = 10$. The inner products are $x_1^\top x_1 = 5$, $x_2^\top x_2 = 10$, $x_1^\top x_2 = 5$. The dual objective becomes:

$$\max_{\alpha_1,\alpha_2} \alpha_1 + \alpha_2 - \frac{1}{2}(5\alpha_1^2 + 10\alpha_2^2 - 10\alpha_1\alpha_2)$$

subject to $\alpha_1 - \alpha_2 = 0$ and $0 \leq \alpha_1, \alpha_2 \leq 10$. Solving $\alpha_1 = \alpha_2$ gives:

$$\max_{\alpha} 2\alpha - \frac{1}{2}(5\alpha^2 + 10\alpha^2 - 10\alpha^2) = 2\alpha - \frac{5}{2}\alpha^2.$$

The maximum occurs at $\alpha = 0.4$ (within bounds), yielding objective value 0.8. The weights are $w = 0.4 \cdot 1 \cdot (1,2) + 0.4 \cdot (-1) \cdot (3,1) = (-0.8, 0.4)$.

Dual Problem Formulation

- Objective function reconstruction: The dual problem replaces the weight vectors and bias terms in the original objective function with functions of Lagrange multipliers, thereby transforming the quadratic programming problem into a convex optimization problem that relies only on multipliers and improving computational efficiency.

- Kernel Trick Compatibility: The dual form naturally supports kernel functions, which extend linear SVM to nonlinear separable data through inner product operations without the need for explicit computation of high-dimensional feature maps.

- Sparsity manifestation: The solution of the dual problem has sparsity, with most multipliers being zero, and only the corresponding multipliers of the support vector being non-zero, which greatly reduces the storage and computational overhead in the prediction stage.

- Simplification of Constraints: The dual problem transforms the original inequality constraints into simple boundary constraints (such as $0 \leqslant \alpha_i \leqslant C$), making it easier to efficiently solve using algorithms such as Sequential Minimum Optimization (SMO).

Lagrange Multipliers

- Optimization problem transformation: Lagrange multiplier method simplifies the solving process by introducing Lagrange multipliers to transform the constrained original optimization problem into an unconstrained Lagrange function. This method is particularly suitable for handling inequality constraints in SVM, such as maximizing classification intervals.

- KKT condition: The Karush Kuhn Tucker condition is the core of the Lagrange multiplier method, used to determine the existence of the optimal solution. In SVM, the KKT condition requires that the multipliers corresponding to the support vector are non-zero, while the multipliers of other samples are zero, ensuring the uniqueness and sparsity of the solution.

- Duality analysis: Through the Lagrange duality problem, the original high-dimensional optimization problem can be transformed into a more easily solvable dual form. The dual problem not only reduces computational complexity, but also reveals the crucial role of support vectors (only support vectors affect the classification hyperplane).

Karush-Kuhn-Tucker (KKT) conditions of SVM

- The Karush-Kuhn-Tucker (KKT) conditions for a Support Vector Machine (SVM) are necessary and sufficient conditions for optimality in convex optimization.

Given the primal SVM optimization problem (with soft margin):

Minimize:

$\frac{1}{2}\|\mathbf{w}\|^2 + C\sum_{i=1}^{n}\xi_i$

Subject to:

$y_i(\mathbf{w}\cdot\mathbf{x}_i + b) \geq 1 - \xi_i,$

$\xi_i \geq 0 \quad \forall i$

The Lagrangian is:

$$L(\mathbf{w}, b, \boldsymbol{\xi}, \boldsymbol{\alpha}, \boldsymbol{\mu}) = \frac{1}{2}\|\mathbf{w}\|^2 + C\sum_{i=1}^{n}\xi_i - \sum_{i=1}^{n}\alpha_i\left[y_i(\mathbf{w}\cdot\mathbf{x}_i + b) - 1 + \xi_i\right] - \sum_{i=1}^{n}\mu_i\xi_i$$

Karush-Kuhn-Tucker (KKT) Conditions of SVM

1. **Stationarity:**

$$\nabla_{\mathbf{w}} L = \mathbf{w} - \sum_{i=1}^{n} \alpha_i y_i \mathbf{x}_i = 0$$
$$\frac{\partial L}{\partial b} = -\sum_{i=1}^{n} \alpha_i y_i = 0$$
$$\frac{\partial L}{\partial \xi_i} = C - \alpha_i - \mu_i = 0$$

2. **Primal Feasibility:**

$$y_i(\mathbf{w} \cdot \mathbf{x}_i + b) \geq 1 - \xi_i$$
$$\xi_i \geq 0$$

3. **Dual Feasibility:**

$$\alpha_i \geq 0, \quad \mu_i \geq 0$$

4. **Complementary Slackness:**

$$\alpha_i \left[y_i(\mathbf{w} \cdot \mathbf{x}_i + b) - 1 + \xi_i \right] = 0$$
$$\mu_i \xi_i = 0$$

From $\mu_i = C - \alpha_i$, the conditions simplify to:

- $\mathbf{w} = \sum_i \alpha_i y_i \mathbf{x}_i$
- $\sum_i \alpha_i y_i = 0$
- $0 \leq \alpha_i \leq C$
- $\alpha_i [y_i(\mathbf{w} \cdot \mathbf{x}_i + b) - 1 + \xi_i] = 0$
- $(C - \alpha_i)\xi_i = 0$

Example : Karush-Kuhn-Tucker (KKT) conditions of SVM

Consider a linearly separable dataset with two points:

- Positive class: $(1, 1)$, label $y_1 = +1$
- Negative class: $(0, 0)$, label $y_2 = -1$

Assume C is large (hard margin). The optimal hyperplane is $x_1 - x_2 = 0$, with margin $1/\sqrt{2}$. The support vectors are $(1, 1)$ and $(0, 0)$.

Solve the dual problem:

$$\max_{\alpha} \sum_i \alpha_i - \frac{1}{2} \sum_i \sum_j \alpha_i \alpha_j y_i y_j \mathbf{x}_i \cdot \mathbf{x}_j$$

Subject to:
$\alpha_i \geq 0, \quad \sum_i \alpha_i y_i = 0$

Compute kernels:

- $\mathbf{x}_1 \cdot \mathbf{x}_1 = 2, \quad \mathbf{x}_2 \cdot \mathbf{x}_2 = 0, \quad \mathbf{x}_1 \cdot \mathbf{x}_2 = 0$

Dual objective:

$$\alpha_1 + \alpha_2 - \frac{1}{2}(2\alpha_1^2 + 0)$$

With constraint: $\alpha_1 - \alpha_2 = 0 \Rightarrow \alpha_1 = \alpha_2$

Example : Karush-Kuhn-Tucker (KKT) conditions of SVM

Set $\alpha_1 = \alpha_2 = t$, then:

$$2t - \frac{1}{2}(2t^2) = 2t - t^2$$

Maximize: derivative $2 - 2t = 0 \Rightarrow t = 1$. So $\alpha_1 = \alpha_2 = 1$.

Compute $\mathbf{w} = \sum_i \alpha_i y_i \mathbf{x}_i = 1 \cdot (+1) \cdot (1,1) + 1 \cdot (-1) \cdot (0,0) = (1,1)$

Compute b from any support vector, e.g., for $(1,1)$:

$y_1(\mathbf{w} \cdot \mathbf{x}_1 + b) = 1 \Rightarrow (1,1) \cdot (1,1) + b = 2 + b = 1 \Rightarrow b = -1$

Check KKT:

- Stationarity: $\mathbf{w} = (1,1) = \sum \alpha_i y_i \mathbf{x}_i$ ✓
- Dual feasibility: $\alpha_i = 1 \geq 0$ ✓
- Primal feasibility:
  For $(1,1)$: $1 \cdot (2 - 1) = 1 \geq 1$ ✓
  For $(0,0)$: $-1 \cdot (0 - 1) = 1 \geq 1$ ✓
- Complementary slackness:
  $\alpha_i > 0 \Rightarrow y_i(\mathbf{w} \cdot \mathbf{x}_i + b) = 1$ ✓

All KKT conditions are satisfied.

SMO Algorithm



Partition optimization strategy

The SMO algorithm decomposes large-scale quadratic programming problems into multiple small-scale sub problems by optimizing only one pair of Lagrange multipliers at a time (fixing the other multipliers), significantly reducing computational complexity.



Heuristic selection rule

The selection of multiplier pairs is based on heuristic rules (such as prioritizing the sample with the most severe violation of KKT conditions), which accelerates algorithm convergence, while combining analytical solutions to ensure efficient and accurate iteration at each step.



Convergence guarantee

The SMO algorithm converges to the global optimal solution within a finite number of steps, making it suitable for large-scale datasets and becoming the actual standard algorithm for training SVM (such as the implementation core of LibSVM).

SMO Algorithm

- The key steps include:
  - Initialization of α and b.
  - Selection of two multipliers violating **Karush-Kuhn-Tucker (KKT)** conditions.
  - Analytical solution for the two-variable subproblem with box constraints $0 \leq \alpha_i \leq C$.
  - Update of threshold b and error cache.
  - Convergence check based on KKT satisfaction or maximum iterations.
- SMO avoids complex quadratic solvers by leveraging analytical updates, significantly accelerating SVM training for large datasets.

05

Practical Applications

Text Classification

Junk email filtering

Support Vector Machine (SVM) can effectively distinguish spam emails from normal emails by constructing a high-dimensional feature space. Its classification accuracy is high and it has strong robustness to noisy data.

Emotion analysis

SVM can be used to analyze the emotional tendencies of text (such as positive, negative, or neutral), using feature extraction methods such as word frequency and TF-IDF, combined with kernel functions to optimize nonlinear classification boundaries.

News classification

SVM can classify news articles into multiple categories (such as politics, sports, technology), utilizing the semantic and statistical features of the text to achieve high-precision topic segmentation.

Image Recognition

Handwritten digit recognition

SVM performs well on datasets such as MNIST. By extracting pixel grayscale values or HOG features, combined with RBF kernel function to handle nonlinear separable problems, the recognition accuracy can reach over 98%.

Facial detection

By combining SVM with Haar or LBP features, an efficient facial detection model can be constructed to distinguish between facial and non facial regions, which is widely used in security and camera focusing systems.

Medical imaging analysis

SVM can be used for lesion classification in X-ray and MRI images (such as tumor benign and malignant judgment), and assists in diagnostic decision-making through texture and morphological feature extraction.

Object recognition

In complex backgrounds, SVM can achieve specific object localization and classification through SIFT or SURF feature matching, such as traffic sign recognition or industrial part quality inspection.

Bioinformatics



Gene expression data analysis

SVM can identify cancer-related gene markers and predict disease subtypes by processing high-dimensional features of microarray data, assisting in personalized treatment planning.

Protein structure prediction

SVM combines the physical and chemical properties of amino acid sequences (such as hydrophobicity and charge) to predict protein secondary structures (alpha helices, beta folds) with higher accuracy than traditional statistical methods.

Drug activity screening

SVM models can classify the biological activity of compounds based on molecular descriptors such as LogP and molar refractive index, accelerating the virtual screening process in drug discovery.

06

Tools and Libraries

LIBSVM

- Efficient Algorithm Implementation: LIBSVM is an efficient library specifically designed for Support Vector Machines (SVM), written in C++, supporting classification, regression, and anomaly detection tasks. Its core algorithm optimizes the training speed of large-scale datasets and provides a variety of kernel functions (such as linear, polynomial, RBF, and sigmoid) for selection.

- Multilingual interface support: LIBSVM not only provides the C++interface, but also supports the encapsulation of Python, Java, MATLAB and other programming languages, which is convenient for developers to call in different environments. Its Python interface is implemented through the libsvm package, simplifying the data preprocessing and model evaluation process.

- Rich documentation and community: LIBSVM comes with a detailed user guide and example code, covering comprehensive content from data formats to parameter explanations. An active academic community continuously updates frequently asked questions (FAQ) to help users solve practical application problems.

Scikit-learn Implementation

- Integrated machine learning tool: Scikit learn's SVM module provides a complete implementation of SVM, including SVM (classification), SVR (regression), and OneClassSVM (anomaly detection). Its API design is consistent with other Scikit learn components, supporting pipeline and cross validation.

- Flexible data processing: Supports sparse matrix input, suitable for high-dimensional data scenarios such as text classification. Built in data standardization and feature scaling functions, users can use tools such as StandardScaler to preprocess data and improve model performance.

- Kernel function and parameter configuration: providing linear RBF、 Polynomial kernel function, and allows custom kernel functions. Key parameters such as' C '(regularization coefficient) and' gamma '(kernel function bandwidth) can be optimized through GridSearchCV.

- Parallel computing support: Utilizing the 'n_jobs' parameter to achieve multi-threaded training and accelerate large-scale dataset processing. In addition, it supports incremental learning (partial-fit) to adapt to memory limited scenarios.

Parameter Tuning Guide

- The role of the regularization parameter : 'C' is to control the model's tolerance for misclassified samples. A smaller 'C' value tends to maximize the interval but allows for more classification errors, making it suitable for data with more noise; Strict classification of larger 'C' values may result in overfitting. Suggest searching within the range of '[0.1, 10]' through cross validation.

- Kernel function selection strategy: Linear kernels are suitable for high-dimensional sparse data (such as text); RBF kernel is suitable for nonlinear separable problems, but requires adjusting the gamma parameter; The polynomial kernel needs to specify 'degree', and the complexity increases with the order.

- The influence of gamma parameter: gamma determines the locality of RBF kernel, and the larger the value, the more sensitive the model is to a single sample. It is usually recommended to use 'scale' (1/(n_features X. var())) or 'auto' (1/n_features) as the initial value, and then fine tune it through grid search.

07

Strengths, and Limitations

Discuss its robustness to noise and outlier resilience due to margin maximization

Margin Maximization

SVM maximizes the margin between classes, creating a robust decision boundary. This large separation reduces sensitivity to small variations in data, enhancing resilience against noise.

Outlier Insensitivity

Due to soft margin and slack variables, SVM tolerates misclassified outliers near the boundary. The focus on support vectors limits their influence on the final model.

Generalization Strength

By prioritizing wide margins, SVM improves generalization performance. This makes it effective in real-world scenarios where data often contains noise and anomalies.

Key drawbacks including long training times on large-scale datasets

Training Time

SVM training time increases significantly with large datasets due to solving a quadratic optimization problem. This makes it less scalable for big data applications compared to simpler models like logistic regression or decision trees.

Memory Use

SVM requires storing a kernel matrix that grows quadratically with the number of samples, leading to high memory consumption. This limits its use in resource-constrained or real-time environments with massive data inputs.

Parameter Sensitivity

Performance heavily depends on proper tuning of C and kernel parameters, which can be time-consuming. Poor choices lead to overfitting or underfitting, especially on complex or noisy large-scale datasets.

Address sensitivity to parameter tuning and kernel selection challenges

Tuning C

The regularization parameter C controls the trade-off between margin width and classification error. Selecting an inappropriate C can lead to either underfitting or overfitting, requiring careful cross-validation for optimal results.





Kernel Choice

Choosing the right kernel is crucial for performance, as it defines the feature space transformation. Poor selection may result in poor generalization, especially on complex or high-dimensional datasets.

Hyperparameter Search

Finding optimal parameters like C and gamma often involves grid or random search, which can be computationally expensive. Efficient search strategies are essential to reduce training time and improve model quality.



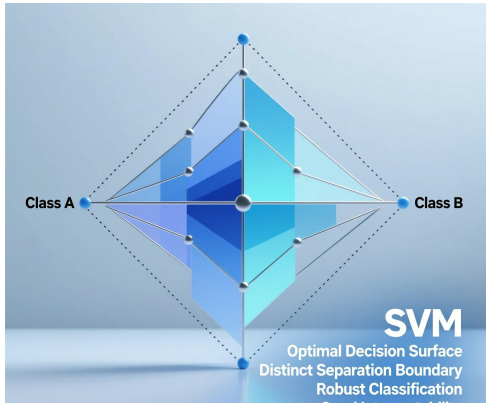


Model Sensitivity

SVM performance is highly sensitive to parameter settings and kernel type. Small changes can significantly impact accuracy, making expert knowledge or automated tuning vital for reliable deployment.

When to choose SVM over other machine learning models





High-Dim Data

SVM excels in high-dimensional spaces, making it ideal for text and image data. Its performance remains strong even when features exceed samples.

Small Samples

SVM performs well with limited training data. It generalizes effectively due to margin maximization, unlike deep models needing large datasets.

Clear Margin

When classes have a distinct separation boundary, SVM finds optimal decision surfaces. This leads to robust classification with good interpretability.

Nonlinear Kernels

With kernel trick, SVM handles complex patterns efficiently. Choose RBF or polynomial kernels when data isn't linearly separable.

PART 01

Introduction to PCA

Definition and Basic Concepts

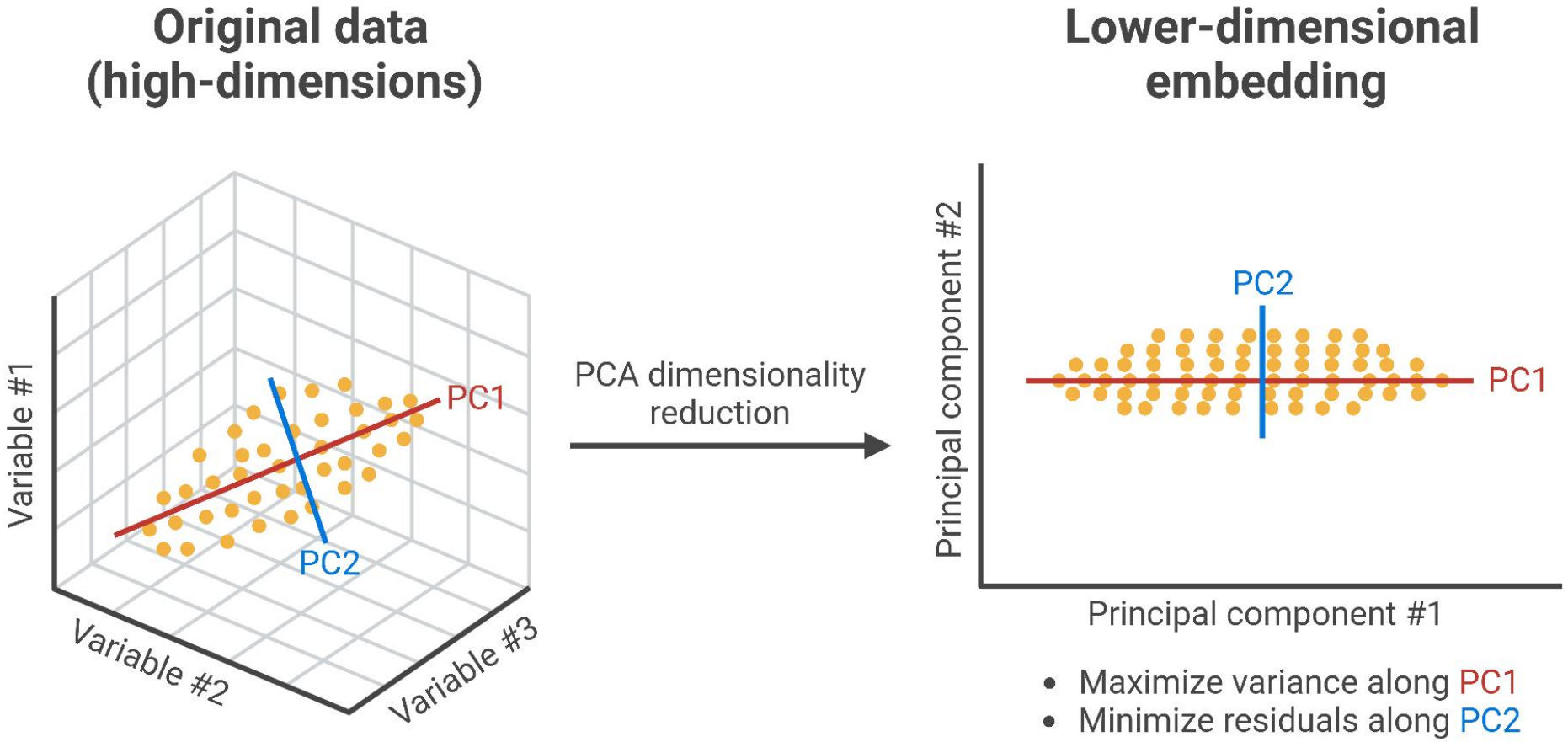
Principal Component Analysis (PCA) : The core goal of PCA is to **transform high-dimensional data into a lower dimensional space through mathematical transformations, while preserving important information** from the original data as much as possible. This is also one of the reasons why PCA is widely used.

Mathematical Concepts: Refers to concepts such as **standard deviation, covariance, eigenvectors, and eigenvalues,** used to quantify data features and reveal relationships between variables. They are the foundation for understanding PCA and its implementation.

Pattern Recognition: is a statistical technique that can be applied in fields such as **facial recognition and image compression.** Its aim is to extract key information from high-dimensional data and reveal data structure and variation patterns.



PCA


Original data
(high-dimensions)
Variable #1
Variable #2
Variable #3
PC1
PC2
PCA dimensionality
reduction
Lower-dimensional
embedding
Principal component #2
Principal component #1
PC2
PC1
• Maximize variance along PC1
• Minimize residuals along PC2

Historical Development

Origins of PCA

PCA originated from statistics and its development can be traced back to the early 20th century. Many scholars, such as Karl Pearson and Harold Hotelling, made contributions to it, gradually forming the analytical methods known today.

Evolution of PCA

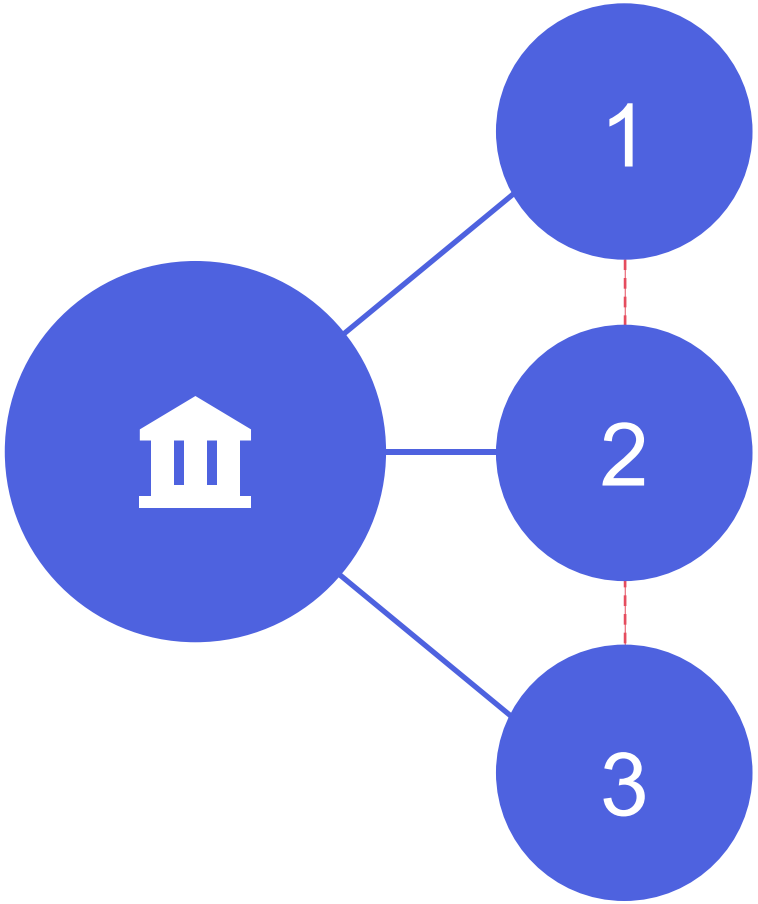
With the development of technology and the increasing demand for data analysis, PCA has evolved from a simple tool to an important component in complex systems, widely used in various fields such as biology, psychology, engineering, etc.

Modern Applications

PCA continues to play an important role in modern society, not only in the field of scientific research, but also in business analysis, financial engineering, medical diagnosis, and other areas, demonstrating enormous potential and broad application prospects.



Importance in Data Analysis



Data Reduction

Through PCA, high-dimensional data can be simplified, redundant information can be reduced, and analysis efficiency and accuracy can be improved. It is an indispensable part of data processing and analysis, providing strong support for decision-making.

Noise Reduction

During the data collection process, PCA can effectively reduce noise interference, highlight key information points, optimize data analysis results, improve data quality and reliability, and provide strong support for decision-making.

Comparative Analysis

By comparing data from different groups, PCA can reveal the differences and similarities between groups, providing valuable references for in-depth research and promoting a deeper understanding of complex datasets.

PART 02

Mathematical Foundations

Covariance and Correlation

Standard deviation and covariance: Standard deviation measures the degree of data dispersion, while covariance reflects the linear relationship between the trends of two variables.

Clever application of covariance matrix: The covariance matrix aggregates all covariance information, making it easy to analyze the relationships between variables in complex datasets.

Correlation coefficient: After standardizing the data, calculate the covariance to obtain the correlation coefficient, which measures the strength and direction of linear correlation between variables.

Correlation Matrix: Construct a correlation matrix to comprehensively display the linear correlation between variables in the dataset, providing a foundation for in-depth analysis.

Covariance and Correlation

1. Covariance

Population Covariance

$$\mathrm{Cov}(X,Y) = \frac{1}{N}\sum_{i=1}^{N}(X_i - \mu_X)(Y_i - \mu_Y)$$

Sample Covariance

$$\mathrm{cov}(x,y) = \frac{1}{n-1}\sum_{i=1}^{n}(x_i - \bar{x})(y_i - \bar{y})$$

Where:

- μ_X, μ_Y: population means
- $\bar{x}, \bar{y}$: sample means
- N or n: number of paired observations

2. Pearson Correlation Coefficient

Population Correlation

$$\rho_{XY} = \frac{\mathrm{Cov}(X,Y)}{\sigma_X \sigma_Y}$$

Sample Correlation

$$r_{xy} = \frac{\mathrm{cov}(x,y)}{s_x s_y}$$

Where:

- σ_X, σ_Y: population standard deviations
- s_x, s_y: sample standard deviations



Example of Covariance and Correlation

Let:

$$\mathbf{x} = (2, 4, 5, 6, 8), \quad \mathbf{y} = (3, 5, 4, 6, 7)$$

Step 1: Means

$$\bar{x} = \frac{1}{5}\sum_{i=1}^{5} x_i = 5, \quad \bar{y} = \frac{1}{5}\sum_{i=1}^{5} y_i = 5$$

Step 2: Sample Covariance

$$\text{cov}(x, y) = \frac{1}{n-1}\sum_{i=1}^{n}(x_i - \bar{x})(y_i - \bar{y}) = \frac{1}{4} \cdot 13 = 3.25$$

Step 3: Sample Standard Deviations

$$s_x = \sqrt{\frac{1}{n-1}\sum_{i=1}^{n}(x_i - \bar{x})^2} = \sqrt{5}, \quad s_y = \sqrt{\frac{1}{n-1}\sum_{i=1}^{n}(y_i - \bar{y})^2} = \sqrt{2.5}$$

Step 4: Sample Correlation

$$r_{xy} = \frac{\text{cov}(x, y)}{s_x s_y} = \frac{3.25}{\sqrt{5} \cdot \sqrt{2.5}} \approx 0.919$$

Interpretation

- Covariance: $\text{cov}(x, y) = 3.25 \Rightarrow$ positive joint variability
- Correlation: $r_{xy} \approx 0.919 \Rightarrow$ strong positive linear relationship



Eigenvalues and Eigenvectors

Eigenvalues and eigenvectors: Eigenvalues reflect the components that remain unchanged in direction during transformation, while eigenvectors correspond to these unchanged directions.

Matrix diagonalization: By decomposing the eigenvalues, the matrix is transformed into a more concise form composed of eigenvectors, revealing the internal structure of the matrix.

Principal Component Extraction: In PCA, the most important principal components in the dataset are identified and extracted using eigenvalues and eigenvectors.

Contribution rate and cumulative contribution rate: Eigenvalues represent the contribution rates of each principal component, while cumulative eigenvalues reflect the completeness of information retention.

Eigenvalues and Eigenvectors

1. Definition and Formula

For a square matrix A, an **eigenvector** $\mathbf{v}$ and its corresponding **eigenvalue** λ satisfy:

$$A\mathbf{v} = \lambda\mathbf{v}, \quad \mathbf{v} \neq \mathbf{0}$$

Steps to compute eigenvalues and eigenvectors:

1. **Find eigenvalues** by solving the **characteristic equation**:

$$\det(A - \lambda I) = 0$$

where I is the identity matrix.

2. **Find eigenvectors** for each eigenvalue λ by solving:

$$(A - \lambda I)\mathbf{v} = \mathbf{0}$$



Example of Eigenvalues and Eigenvectors

Let's find the eigenvalues and eigenvectors of the matrix:

$$A = \begin{bmatrix} 4 & 1 \\ 2 & 3 \end{bmatrix}$$

Step 1: Find Eigenvalues

Solve the characteristic equation:

$$\det(A - \lambda I) = \det \begin{bmatrix} 4-\lambda & 1 \\ 2 & 3-\lambda \end{bmatrix} = 0$$

Compute the determinant:

$$(4 - \lambda)(3 - \lambda) - (2)(1) = 0$$

$$\lambda^2 - 7\lambda + 10 = 0$$

Solve the quadratic equation:

$$\lambda = \frac{7 \pm \sqrt{49 - 40}}{2} = \frac{7 \pm 3}{2}$$

Eigenvalues:

$$\lambda_1 = 5, \quad \lambda_2 = 2$$

Example of Eigenvalues and Eigenvectors

Step 2: Find Eigenvectors

For $\lambda_1 = 5$:

Solve $(A - 5I)\mathbf{v} = \mathbf{0}$:

$$\begin{bmatrix} -1 & 1 \\ 2 & -2 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

From the first row:

$$-x + y = 0 \implies y = x$$

Eigenvector for $\lambda = 5$:

$$\mathbf{v}_1 = \begin{bmatrix} 1 \\ 1 \end{bmatrix} \quad (\text{or any scalar multiple, e.g., } k \begin{bmatrix} 1 \\ 1 \end{bmatrix}, k \neq 0)$$

For $\lambda_2 = 2$:

Solve $(A - 2I)\mathbf{v} = \mathbf{0}$:

$$\begin{bmatrix} 2 & 1 \\ 2 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

From the first row:

$$2x + y = 0 \implies y = -2x$$

Eigenvector for $\lambda = 2$:

$$\mathbf{v}_2 = \begin{bmatrix} 1 \\ -2 \end{bmatrix} \quad (\text{or any scalar multiple, e.g., } k \begin{bmatrix} 1 \\ -2 \end{bmatrix}, k \neq 0)$$

3. Final Answer

- **Eigenvalues:** $\lambda_1 = 5, \lambda_2 = 2$
- **Corresponding Eigenvectors:**
  - For $\lambda = 5$: $\mathbf{v}_1 = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$
  - For $\lambda = 2$: $\mathbf{v}_2 = \begin{bmatrix} 1 \\ -2 \end{bmatrix}$

Singular Value Decomposition

SVD decomposition: Decompose a matrix into the sum of three parts: U, ∑, and V *, where U and V * are the left and right singular matrices, respectively, and ∑ is the diagonal matrix of singular values.

Compression and noise reduction: By retaining the components corresponding to large singular values, data compression and noise reduction are achieved, improving data quality and subsequent analysis efficiency.

Contribution ranking: Singular values reflect the importance of components, and sorting by size can determine the order of principal components, providing guidance for data analysis.

Widely used: SVD has a wide range of applications in signal processing, statistics, machine learning, and other fields, and is an effective tool for processing complex data.

SVD Formula

SVD breaks any **real matrix** A (size $m \times n$) into **three matrices**:

$$A = U\Sigma V^T$$

Where:

- **U** is an $m \times m$ **orthogonal matrix** (columns = **left singular vectors**),
- **Σ** is an $m \times n$ **diagonal matrix** with **non-negative singular values** on the diagonal, sorted in **descending order**,
- **V^T** is an $n \times n$ **orthogonal matrix** (rows = **right singular vectors**, transposed).



SVD Example

Let's say:

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$$

Then SVD gives:

$$U = \begin{bmatrix} -0.404 & -0.914 \\ -0.914 & 0.404 \end{bmatrix}, \quad \Sigma = \begin{bmatrix} 5.465 & 0 \\ 0 & 0.366 \end{bmatrix}, \quad V^T = \begin{bmatrix} -0.576 & -0.817 \\ -0.817 & 0.576 \end{bmatrix}$$

Now multiply:

$$U\Sigma V^T \approx A$$



This reconstruction is **very close** to the original matrix A, confirming the decomposition is correct.

PART 03

PCA Methodology

Data Standardization

Z-score standardization

Standardization processing is an important step before PCA. By calculating Z-score, the mean of the data is adjusted to 0 and the standard deviation is adjusted to 1, thereby eliminating the influence of dimensionality and highlighting the differences between data.

Data preprocessing

Standardization processing helps improve the effectiveness and accuracy of PCA analysis by eliminating the influence of dimensionality, making the data more comparable and revealing patterns and relationships in the data more accurately.

Principal Components Calculation



Covariance Matrix

Calculating the covariance matrix is the core step of PCA. Through this matrix, we can quantify the linear relationships between dimensions in the dataset, providing a foundation for subsequent component analysis.



Eigenvalues and eigenvectors

Eigenvalues and eigenvectors are key to PCA analysis, representing the variance size and principal direction of the dataset, respectively, helping to understand the main patterns and structures of data changes.



Variance Explained

Variance explained ratio

Variance explained ratio (also known as explained variance ratio) is used to quantify the role of each principal component in explaining the total variance of the data, thereby evaluating the importance of each principal component.

Cumulative variance explained rate

Cumulative variance explained rate refers to the proportion of variance that can be explained by the first m principal components. This indicator helps us understand how many principal components are sufficient.



PCA Algorithm

1.**Center data**: Subtract each feature's mean to ensure zero mean.

2.**Compute covariance matrix**: For centered data matrix $X_{centered}$, calculate $C = \frac{1}{n-1} X_{centered}^T X_{centered}$ (sample covariance).

3.**Eigendecomposition**: Solve $Cv = \lambda v$, where eigenvectors v represent principal directions, and eigenvalues λ quantify variance along each direction.

4.**Sort components**: Rank eigenvectors by descending eigenvalues. The top k eigenvectors form the projection matrix W.

5.**Project data**: Transform data via $Y = X_{centered} W$, where Y contains PC scores.

6.**Variance explained**: The ratio $\frac{\lambda_i}{\sum_j \lambda_j}$ gives the proportion of total variance captured by the i-th PC.



PCA Example

Step 1: Original Data

Let's start with a simple dataset:

$$X = \begin{bmatrix} 1 & 2 \\ 2 & 3 \\ 3 & 4 \\ 4 & 5 \end{bmatrix}$$

Step 2: Center the Data

Compute the mean of each column:

- Mean of column 1: $\frac{1+2+3+4}{4} = 2.5$
- Mean of column 2: $\frac{2+3+4+5}{4} = 3.5$

Subtract the means:

$$X_{\text{centered}} = \begin{bmatrix} -1.5 & -1.5 \\ -0.5 & -0.5 \\ 0.5 & 0.5 \\ 1.5 & 1.5 \end{bmatrix}$$

Step 3: Compute the Covariance Matrix

Use the formula:

$$C = \frac{1}{n-1} X_{\text{centered}}^T X_{\text{centered}}$$

$$C = \frac{1}{3} \begin{bmatrix} 5 & 5 \\ 5 & 5 \end{bmatrix} = \begin{bmatrix} 1.667 & 1.667 \\ 1.667 & 1.667 \end{bmatrix}$$

$$X_{\text{centered}}^T X_{\text{centered}} = \begin{bmatrix} -1.5 & -0.5 & 0.5 & 1.5 \\ -1.5 & -0.5 & 0.5 & 1.5 \end{bmatrix} \begin{bmatrix} -1.5 & -1.5 \\ -0.5 & -0.5 \\ 0.5 & 0.5 \\ 1.5 & 1.5 \end{bmatrix} = \begin{bmatrix} 5 & 5 \\ 5 & 5 \end{bmatrix}$$

$$C = \frac{1}{3} \begin{bmatrix} 5 & 5 \\ 5 & 5 \end{bmatrix} = \begin{bmatrix} 1.6667 & 1.6667 \\ 1.6667 & 1.6667 \end{bmatrix}$$

PCA Example

Step 4: Eigendecomposition

Find eigenvalues and eigenvectors of the covariance matrix:

- Eigenvalues:

$$\lambda_1 = 3.333, \quad \lambda_2 = 0$$

$$\lambda_1 = 1.6667 + 1.6667 = 3.3333$$

$$\lambda_2 = 1.6667 - 1.6667 = 0$$

$$\det(C - \lambda I) = 0$$

$$\det \begin{bmatrix} 1.6667 - \lambda & 1.6667 \\ 1.6667 & 1.6667 - \lambda \end{bmatrix} = 0$$

$$(1.6667 - \lambda)^2 - (1.6667)^2 = 0$$

$$(1.6667 - \lambda)^2 = (1.6667)^2 \quad \Rightarrow \quad 1.6667 - \lambda = \pm 1.6667$$

- Eigenvectors:

$$v_1 = \begin{bmatrix} 0.707 \\ 0.707 \end{bmatrix}, \quad v_2 = \begin{bmatrix} -0.707 \\ 0.707 \end{bmatrix}$$

$\lambda_1 = 3.3333$:

$$(C - 3.3333I)v = 0 \quad \Rightarrow \quad \begin{bmatrix} -1.6666 & 1.6667 \\ 1.6667 & -1.6666 \end{bmatrix} v = 0$$

Step 5: Project Data onto the First Principal Component

Project the centered data onto the first eigenvector:

$$Y = X_{\text{centered}} \cdot v_1 = \begin{bmatrix} -2.12 \\ -0.71 \\ 0.71 \\ 2.12 \end{bmatrix}$$

$$Y = X_{\text{centered}} \cdot v_1$$

$$Y = \begin{bmatrix} -1.5 & -1.5 \\ -0.5 & -0.5 \\ 0.5 & 0.5 \\ 1.5 & 1.5 \end{bmatrix} \begin{bmatrix} 0.7071 \\ 0.7071 \end{bmatrix} = \begin{bmatrix} -2.1213 \\ -0.7071 \\ 0.7071 \\ 2.1213 \end{bmatrix}$$

$$Y \approx [-2.12, -0.71, 0.71, 2.12]^T$$

Step 6: Variance Explained

- Total variance = $\lambda_1 + \lambda_2 = 3.333 + 0 = 3.333$
- Variance explained by PC1:

$$\frac{3.333}{3.333} = 100\%$$

Interpretation:

- The first principal component captures 100% of the total variance because the second eigenvalue is zero.
- This indicates perfect linear correlation between the two original variables.
- Geometrically, all data points lie on a straight line, so one component is enough to describe the data completely.

PART 04

Applications of PCA



Dimensionality Reduction

Pattern Recognition

By reducing the dimensionality of the data, PCA helps in recognizing patterns that are difficult to discern in the original high-dimensional space.

Data Compression

PCA allows for the compression of data while retaining most of the significant information, making it a useful technique for applications where data storage is limited.

Noise Reduction

In noisy data, PCA can help to reduce the dimensionality, thus reducing the influence of random noise while preserving the essential features of the data.

Visualization

Dimensionality reduction through PCA facilitates the visualization of data in lower dimensions, making it easier to interpret and gain insights from complex datasets.

Data Visualization

Scatter Plots

PCA enables the creation of scatter plots of the first few principal components, which can reveal clusters, correlations, and other patterns that are not immediately apparent in the original data.

Heat Maps

By using biplots or loading plots, PCA allows for the visualization of the contribution of each variable to the principal components, providing insights into the data structure in a graphical manner.

Principal Coordinates

Principal coordinates are derived from PCA and represent points in a low-dimensional space where distances reflect similarity in the original high-dimensional data.

Biplots

Biplots are a type of visualization that displays both the original variables and the principal components on the same plot, providing a comprehensive view of the data structure.

Feature Extraction

01

Correlation

PCA can be used to extract features that are correlated with a particular response variable, such as predicting stock prices based on financial indicators.

02

Classification

In machine learning, PCA can be employed to extract features that are relevant for classification tasks, such as identifying different types of images based on visual characteristics.

03

Clustering

For clustering applications, PCA can be utilized to extract features that are suitable for grouping similar observations together, as seen in customer segmentation based on purchasing behavior.

04

Regression

In regression analysis, PCA can assist in extracting features that are predictive of a continuous response variable, such as predicting house prices based on various house characteristics.



PART 05

Advanced Topics

Kernel PCA

Kernel PCA can handle nonlinear problems by using kernel techniques to map raw data to a high-dimensional feature space, making it linearly separable and enabling efficient nonlinear principal component analysis.

Flexibility and efficiency: Cleverly combining kernel technology with PCA, it not only maintains the flexibility of PCA, but also significantly improves the ability to handle nonlinear data, providing new perspectives and tools for complex data analysis.



Kernel PCA

Kernel PCA extends PCA by mapping data to a high-dimensional feature space via a kernel function $\kappa(x_i, x_j) = \langle \Phi(x_i), \Phi(x_j) \rangle$, where Φ is an implicit mapping. The centered kernel matrix $\bar{K}$ is computed as:

$$\bar{K} = K - 1_N K - K 1_N + 1_N K 1_N,$$

where $K_{ij} = \kappa(x_i, x_j)$ and 1_N is an $N \times N$ matrix with all elements $1/N$. The principal components are obtained by solving the eigenvalue problem:

$$\bar{K}\alpha = \lambda\alpha,$$

and projecting data onto the top k eigenvectors.



Kernel PCA Example– Polynomial Kernel (degree-2)

Let Data set:

Data set = $\{\mathbf{x}_1 = (1,2)^T, \mathbf{x}_2 = (3,4)^T\}$ and define the kernel function $\kappa(\mathbf{x}, \mathbf{y}) = (\mathbf{x}^T\mathbf{y} + 1)^2$.

1. Kernel Matrix Construction

The kernel matrix $K \in \mathbb{R}^{2\times2}$ is computed as:

$\kappa(x_1, x_1) = (1 + 4 + 1)^2 = 36$

$\kappa(x_1, x_2) = \kappa(x_2, x_1) = (3 + 8 + 1)^2 = 144$

$\kappa(x_2, x_2) = (9 + 16 + 1)^2 = 676$

Thus, $K = \begin{bmatrix} 36 & 144 \\ 144 & 676 \end{bmatrix}$.

2. Kernel Matrix Centering

The centered kernel matrix $\bar{K}$ is given by:

$$\bar{K} = K - 1_N K - K 1_N + 1_N K 1_N$$

where 1_N is an N×N matrix with all entries equal to 1/N (here, N=2, so 1/2).

$$1_N K = \frac{1}{2}\begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}\begin{bmatrix} 36 & 144 \\ 144 & 676 \end{bmatrix} = \begin{bmatrix} 90 & 410 \\ 90 & 410 \end{bmatrix}$$

$$K 1_N = (1_N K)^T = \begin{bmatrix} 90 & 90 \\ 410 & 410 \end{bmatrix}$$

$$1_N K 1_N = \frac{1}{2}\begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}\begin{bmatrix} 90 & 90 \\ 410 & 410 \end{bmatrix} = \begin{bmatrix} 250 & 250 \\ 250 & 250 \end{bmatrix}$$

Substituting:

$$\bar{K} = \begin{bmatrix} 36 & 144 \\ 144 & 676 \end{bmatrix} - \begin{bmatrix} 90 & 410 \\ 90 & 410 \end{bmatrix} - \begin{bmatrix} 90 & 90 \\ 410 & 410 \end{bmatrix} + \begin{bmatrix} 250 & 250 \\ 250 & 250 \end{bmatrix} = \begin{bmatrix} 106 & -106 \\ -106 & 106 \end{bmatrix}$$

Kernel PCA Example– Polynomial Kernel (degree-2)

3. Eigen decomposition

The centered matrix $\bar{K}$ is rank-1. Its eigenvalues and eigenvectors are:

$\lambda_1 = 212, \quad \alpha_1 = \frac{1}{\sqrt{2}}\begin{bmatrix} 1 \\ -1 \end{bmatrix}$

$\lambda_2 = 0, \quad \alpha_2 = \frac{1}{\sqrt{2}}\begin{bmatrix} 1 \\ 1 \end{bmatrix}$

(Verification: $\bar{K}\alpha_1 = 212\alpha_1$).

4. Projection of a Novel Point

For a test point $x = (0,0)^T$, the kernel vector is $k = [\kappa(x, x_1), \kappa(x, x_2)]^T = [1, 1]^T$.

Its projection onto the first principal component is:

$$z_1 = \sum_{i=1}^{N} \alpha_{1,i} \cdot \kappa(x, x_i) = \frac{1}{\sqrt{2}}(1) + \frac{-1}{\sqrt{2}}(1) = 0$$

This result indicates the test point resides at the origin in the principal coordinate space.



Key take-aways of Kernel PCA

- The centred kernel matrix $\tilde{K}$ is always symmetric, row- and column-centred, and positive semi-definite (up to numerical error).

- With only two points the feature-space covariance has rank 1, so one non-zero eigenvalue is expected.

- The projection formula uses the centred kernel entries; in code you would apply the same centring weights to the vector $k(x,\cdot)$ automatically.



Robust PCA



Robustness and robustness

Robust PCA aims to improve the model's resistance to outliers and noise by introducing the concepts of robustness and robustness, thereby ensuring the accuracy and reliability of analysis results.



Decomposition and Reconstruction

Decompose the original data matrix into the sum of a low rank matrix and a sparse matrix, where the low rank matrix represents the normal or primary data structure, while the sparse matrix captures outliers or noise, thereby achieving robust data processing.



Robust PCA Algorithm (Augmented Lagrangian Method)

Robust PCA decomposes a matrix X into low-rank (L) and sparse (S) components by solving the optimization problem:

Given a matrix $M \in \mathbb{R}^{m \times n}$ modeled as $M = L + S$, where L is low-rank and S is sparse, the decomposition is formulated as:

$$\min_{L,S} \|L\|_* + \lambda \|S\|_1 \, subject \, to \, L + S = M,$$

with $\|L\|_*$ the nuclear norm, $\|S\|_1 = \sum_{i,j} |S_{ij}|$, and $\lambda = 1/\sqrt{\max(m,n)}$.

Define the augmented Lagrangian:

$$\mathcal{L}(L, S, Y, \mu) = \|L\|_* + \lambda \|S\|_1 + \langle Y, M - L - S \rangle + \frac{\mu}{2} \|M - L - S^2\|_F$$

Iterate until $\|M - L_k - S_k\|_F \leq \varepsilon \|M\|_F$:

1.**L-update**:

$$L_{k+1} = U \cdot soft\left(\Sigma, \mu_k^{-1}\right) \cdot V^\top, where soft(\sigma, \tau) = \max(\sigma - \tau, 0),$$

via SVD of $M - S_k + \mu_k^{-1} Y_k$.

2.**S-update**:

$$S_{k+1} = soft\left(M - L_{k+1} + \mu_k^{-1} Y_k, \lambda/\mu_k\right).$$

3.**Multiplier and penalty update**:

$$Y_{k+1} = Y_k + \mu_k (M - L_{k+1} - S_{k+1}), \, \mu_{k+1} = \min(\rho \mu_k, \mu_{max}).$$



Robust PCA Example

For $M = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 20 \end{bmatrix}$, with $\lambda = 1/\sqrt{3}$, the solution converges to:

$$L = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}, S = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 11 \end{bmatrix}.$$

Python Implementation

```
import numpy as np, cvxpy as cp
M = np.array([[1,2,3],[4,5,6],[7,8,20]])
L = cp.Variable(M.shape); S = cp.Variable(M.shape)
prob = cp.Problem(cp.Minimize(cp.norm(L,'nuc') + cp.norm(S,1)/np.sqrt(3)), [L + S == M])
prob.solve(solver=cp.SCS)
print("L =\n", L.value); print("S =\n", S.value)
```



Sparse PCA

01

Sparse PCA aims to improve the efficiency and accuracy of analysis by introducing sparsity constraints to select the most important principal components while ignoring those with less contribution or more noise.

02

Variable selection: Similarly, Sparse PCA is also used for variable selection, which involves selecting the most important variables (or features) and ignoring variables with less noise or contribution to improve model performance and prediction accuracy.



Sparse PCA

1. Population objective

   Given the centred sample

$$\mathbf{X} \in \mathbb{R}^{n \times p}, \qquad \sum_{i=1}^{n} x_{ij} = 0 \ \forall j,$$

define the empirical covariance matrix

$$\boldsymbol{\Sigma} = \tfrac{1}{n}\mathbf{X}^{\top}\mathbf{X}.$$

For a single sparse principal component solve

$$\max_{\mathbf{v} \in \mathbb{R}^{p}} \ \mathbf{v}^{\top}\boldsymbol{\Sigma}\mathbf{v} \quad \textbf{subject to} \quad \|\mathbf{v}\|_2 = 1, \ \|\mathbf{v}\|_1 \le t, \ t \ge 0.$$

The penalty form with parameter $\lambda \ge 0$ is

$$\max_{\|\mathbf{v}\|_2 = 1} \ \mathbf{v}^{\top}\boldsymbol{\Sigma}\mathbf{v} - \lambda\|\mathbf{v}\|_1.$$

2. Closed-form solution (rank-1 case)

   Let $\mathbf{u}_1$ be the leading eigenvector of $\boldsymbol{\Sigma}$ (ordinary PCA loading).

   Then the sparse loading is

$$\mathbf{v}^{\star} = \frac{\mathcal{S}_{\lambda}(\mathbf{u}_1)}{\|\mathcal{S}_{\lambda}(\mathbf{u}_1)\|_2}, \qquad \mathcal{S}_{\lambda}(z) = \operatorname{sign}(z) \cdot (|z| - \lambda)_{+}.$$



Sparse PCA Example

Data (already centred)

$$\mathbf{X} = \begin{bmatrix} 2 & 3 & -5 \\ -1 & 1 & 0 \\ 0 & -2 & 2 \\ -1 & 0 & 3 \end{bmatrix}, \qquad \mathbf{\Sigma} = \tfrac{1}{4}\mathbf{X}^\top \mathbf{X} = \begin{bmatrix} 1.5 & 2.0 & -2.5 \\ 2.0 & 3.5 & -5.5 \\ -2.5 & -5.5 & 9.5 \end{bmatrix}.$$

Leading eigenvector (dense)

$$\mathbf{u}_1 \approx \begin{bmatrix} 0.3015 \\ 0.4523 \\ -0.8416 \end{bmatrix}.$$

Choose $\lambda = 0.35$. Soft-threshold and normalize:

$$\mathcal{S}_{0.35}(\mathbf{u}_1) = \begin{bmatrix} 0 \\ 0.1023 \\ 0.4916 \end{bmatrix}, \qquad \mathbf{v}^\star = \begin{bmatrix} 0 \\ 0.2038 \\ 0.9790 \end{bmatrix}.$$

Only the last two coordinates are non-zero; the explained variance is

$$(\mathbf{v}^\star)^\top \mathbf{\Sigma} \mathbf{v}^\star \approx 13.45$$

(97 % of the unrestricted first principal component).

Multiple components of Sparse PCA

For $q > 1$ extract $\mathbf{v}_1^*, \ldots, \mathbf{v}_q^*$ sequentially, deflate

$$\mathbf{X} \leftarrow \mathbf{X} - \mathbf{d}\mathbf{q}^\top, \qquad \mathbf{d} = \mathbf{X}\mathbf{v}^* / \|\mathbf{X}\mathbf{v}^*\|_2,$$

and orthogonalize the loadings; repeat the single-component formula on the deflated matrix.



PART 06

Case Studies

PCA in Genomics

- **Gene expression analysis:** PCA plays a key role in gene expression research by reducing data dimensions, effectively revealing complex relationships between genes, assisting in the discovery of disease biomarkers, and providing support for precision medicine.
- **Genomic association research:** PCA is used in genomic association research to identify gene regions significantly associated with specific diseases or traits by comparing genotype differences between individuals and combining statistical methods.
- **Proteomic analysis:** In proteomics research, PCA assists in data dimensionality reduction processing, clearly distinguishing the differences in protein expression patterns between different samples, and providing a pathway for exploring cellular function and disease mechanisms.
- **Metabolomics research:** PCA technology is widely used in metabolomics research to reduce data dimensions, effectively analyze the interactions between metabolites, and reveal the metabolic pathways and regulatory mechanisms of biological systems.



PCA in Finance

09

Risk management

Financial institutions use PCA technology to accurately quantify operational risks while reducing market risks, ensuring comprehensive financial security and safeguarding the stable operation of business.

10

Investment strategy

Investors analyze market trends based on PCA, develop diversified investment strategies, effectively diversify investment risks, and achieve optimal asset allocation and maximum returns.

11

Financial analysis

Enterprises use PCA to conduct financial health assessments, identify potential financial issues, provide accurate decision support for management, and ensure the stable development of the enterprise.

12

Economic forecasting

Economists use PCA methods to analyze macroeconomic data, predict economic trends, provide scientific basis for government policy-making, and contribute to social stability and prosperity.

PCA in Image Processing

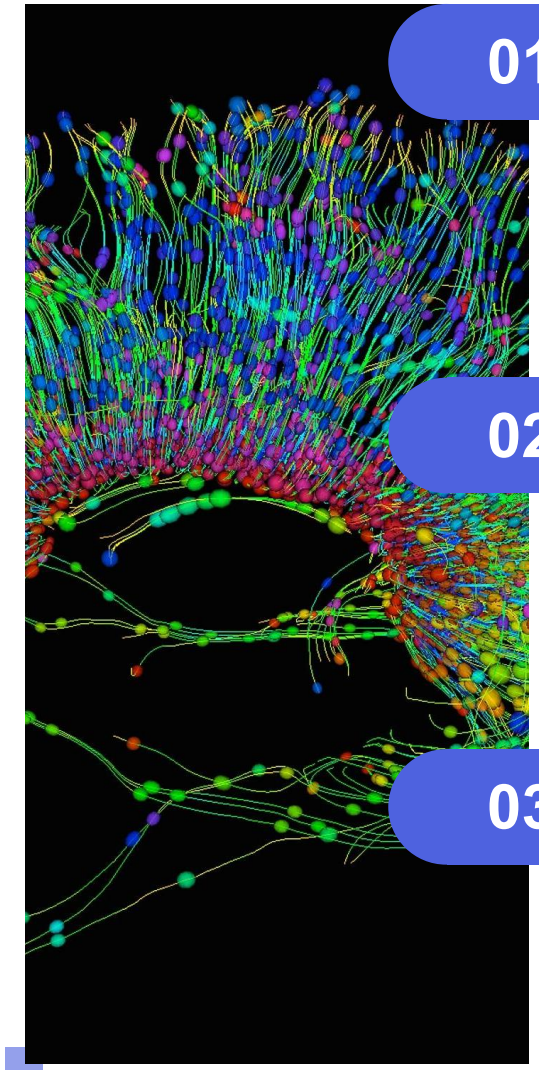
- **Image compression:** PCA technology is integrated into image compression algorithms to effectively reduce image data redundancy, achieve efficient encoding and transmission of images, and improve network image transmission efficiency and clarity.
- **Image processing:** In the field of image processing, PCA method assists edge detection and noise filtering, optimizes image quality, and provides a clear and accurate image foundation for image analysis and recognition.
- **Remote sensing analysis:** PCA is used for remote sensing image analysis to identify key information such as feature points and lines in the image, assist in extracting surface cover attributes, and provide accurate data for geographic research.
- **Medical imaging:** PCA technology is used in medical imaging to automatically detect lesion areas, improve diagnostic accuracy, and reduce manual intervention, achieving more efficient and intelligent medical services.



PART 07

Limitations and Challenges

Sensitivity to Outliers



01

Extreme Value Influence

In PCA analysis, extreme values (also known as outliers) have a significant impact on the results. These abnormal data points can significantly alter the mean and variance of the dataset, thereby affecting the direction and interpretation of principal components.

02

Robustness challenge

In order to obtain robust results, measures need to be taken to handle or clean extreme values, such as data truncation, Winsorization (replacing data with upper and lower boundary values), or using more robust estimation methods.

03

Outlier detection

Before performing PCA, detect and process outliers in the data to ensure the accuracy of the analysis results. Common outlier detection methods include statistical based methods, graphical methods, and machine learning methods.

Interpretability Issues

Factor loading interpretation

In PCA, the interpretation of factor loading (i.e. the correlation coefficient between principal components and original variables) may be affected by the large number of variables, leading to difficulty in interpretation. Excessive variables may increase the complexity of the model.

Rotation and Simplification

Simplifying the structure through variable rotation techniques (such as Varimax rotation) makes the factor load matrix easier to interpret, helping to identify which variables have a significant impact on specific principal components and simplifying model interpretation.

Physical meaning assignment

Ensuring accurate understanding of the physical or practical significance of PCA results requires domain knowledge and appropriate selection of variable rotations, as incorrect rotations may lead to misunderstandings or incorrect applications of the results.

Computational Complexity



Big Data Challenge

As the size of the dataset increases, the computational complexity of PCA significantly increases, and the covariance matrix calculation and resource requirements of large-scale datasets also increase accordingly, which may lead to resource constraints or timeouts.



Matrix operation optimization

For large datasets, optimizing matrix operations to reduce memory usage and computation time is crucial, including the use of sparse matrix representation, iterative solving algorithms, and parallel computing techniques.



Approximation algorithm application

In situations where resources are limited, approximation algorithms can be considered to accelerate the calculation of PCA, such as the randomized PCA algorithm, which quickly estimates principal components by reducing the dimensions of random projections.



PART 08

Future Directions

PCA in Big Data

Challenges faced: With the increasing amount of data, the computational efficiency of PCA on large datasets has become an important challenge. Optimizing algorithms, utilizing parallel computing and approximation techniques have become key to improving processing speed.

Optimization algorithms and parallel computing: For large-scale data, incremental PCA, sparse PCA and other optimization algorithms are adopted, combined with parallel computing technologies such as GPU acceleration, effectively reducing computation time and improving analysis efficiency.

Feature selection and dimensionality reduction: In the era of big data, PCA requires collaborative feature selection techniques to accurately reduce dimensionality and remove redundant information, thereby reducing model complexity and improving the efficiency and accuracy of learning algorithms.

Integration with Machine Learning

01

Deep fusion

PCA and machine learning algorithms are deeply integrated to enhance data expression through feature transformation, providing powerful support for classification, regression and other tasks, and promoting the development of machine learning towards efficiency and accuracy.

02

Kernel Technique Expansion

Introducing kernel techniques, PCA can handle non-linear structured data, enhance its analytical ability on complex datasets, introduce non-linear factors into machine learning models, and improve learning efficiency and generalization ability.

03

Model selection and optimization

Combining cross validation, regularization and other strategies, PCA effectively prevents overfitting and improves the performance of machine learning models. Accurately adjust the complexity of the model to ensure generalization ability and facilitate efficient learning.



Emerging Variants of PCA



Sparse Component Analysis

Introducing sparsity constraints, Sparse Component Analysis effectively identifies important variables, ignores redundant information, simplifies data models, promotes efficient and accurate data processing, and improves analysis efficiency and accuracy.



Kernel Principal Component Analysis

Empowered by kernel techniques, kernel principal component analysis processes non-linear structured data, enhances the ability to analyze complex datasets, helps machine learning models integrate non-linear factors, and improves learning efficiency and generalization ability.



Online learning algorithm

Combining online learning frameworks, online principal component analysis processes data streams in real-time, dynamically adjusts model parameters, ensures that the model is always optimal, and provides real-time and accurate analysis support for rapidly changing environments.

